TRABAJO PRÁCTICO
INVESTIGACIÓN SOBRE APIS



INSTITUTO SUPERIOR PARTICULAR
INCORPORADO N° 4038
"JUAN PABLO II"

TECNICO SUPERIOR EN DESARROLLO DE SOFTWARE

Michael Zarza
Profesora: Gerardo Herrera
Programacion

Índice

1. Introducción..... 3

A. Conceptos Generales y Funcionalidades..... 4

1. Concepto..... 4

2. Historia y evolución..... 4

3. Funcionalidades principales: Métodos, protocolos y formatos de intercambio.. 4

4. Ejemplos prácticos: APIs populares y su impacto en distintas industrias..... 5

B. Usos y Tecnologías a Utilizar..... 5

1. Tecnologías de desarrollo..... 5

2. Lenguajes y herramientas..... 7

3. Implementacion practica..... 7

C. Arquitectura Cliente-Servidor..... 8

1. Modelo de comunicación..... 8

2. Diagrama de arquitectura..... 9

D. Parámetros de seguridad..... 9

1. Requisitos de seguridad..... 9

2. Buenas prácticas..... 10

E. Metodologías de desarrollo, Implantación y Uso..... 11

1. Metodologías de desarrollo..... 11

2. Proceso de Implantación..... 12

3. Casos prácticos de implantación y uso..... 12

Bibliografía..... 15

1. Introducción

Las APIs (Interfaces de Programación de Aplicaciones) son un componente esencial en el desarrollo de software moderno. Permite la comunicación entre diferentes aplicaciones y sistemas, facilitando la interoperabilidad, reutilización de código y escalabilidad. Este trabajo tiene como objetivo profundizar el funcionamiento de las APIs, desde su base teórica hasta la aplicación práctica, analizando sus funcionalidades, casos de uso, tecnologías asociadas, arquitectura cliente-servidor, aspectos de seguridad, metodologías de desarrollo e implantación.

A. Conceptos Generales y Funcionalidades

1. Concepto

Una API (Application Programming Interface) es un conjunto de definiciones, protocolos y herramientas que permiten que diferentes aplicaciones se comuniquen entre sí. Actúa como un intermediario que facilita la interacción y el intercambio de datos y funcionalidades entre sistemas, sin necesidad de conocer la implementación interna de cada uno.

2. Historia y evolución

La historia de las APIs comenzó en los años 60, cuando se crearon para facilitar la reutilización de funciones en los sistemas operativos y bibliotecas de software. Con la llegada de internet, las APIs evolucionaron para convertirse en APIs web, permitiendo la comunicación remota entre aplicaciones a través de protocolos estándar como HTTP.
En la actualidad, las APIs son fundamentales para la construcción de aplicaciones distribuidas, microservicios y la integración entre plataformas, siendo parte esencial de la transformación digital y la innovación tecnológica.
Las APIs modernas son claves en la economía digital, permiten la creación de ecosistemas conectados, fomentan la automatización y hacen posible la interacción entre plataformas diversas.

3. Funcionalidades principales: Métodos, protocolos y formatos de intercambio.

Una API define cómo deben estructurarse las solicitudes y respuestas entre sistemas. Entre las principales características técnicas se destacan:

Métodos HTTP:

- GET: Obtener información
- POST: Enviar información
- PUT: Actualizar información
- DELETE: Eliminar información

Protocolos comunes:

- HTTP/HTTPS: Protocolo más utilizado por las APIs web por su simplicidad y compatibilidad universal.
- SOAP: Protocolo más estricto que opera sobre HTTP, SMTP o FTP.

Formatos de intercambio de datos:

- JSON (JavaScript Object Notation): Ligero, fácil de leer y escribir; es el más usado actualmente.
- XML (eXtensible Markup Language): Más estructurado.
- Otros menos comunes: YAML, CSV, Protobuf.

4. Ejemplos prácticos: APIs populares y su impacto en distintas industrias

- **Google Maps API:** Permite integrar mapas, direcciones y geolocalización en sitios web o apps móviles. Es clave en la industria del transporte, logística y delivery(Uber, Rappi, etc).
- **Twitter API:** Ofrece acceso a publicaciones, perfiles, tendencias y permite crear bots o sistemas de monitoreo en tiempo real. Es ampliamente utilizada en periodismo digital, marketing y análisis de opinión pública.
- **Stripe API:** Utilizada para gestionar pagos electrónicos de forma segura y sencilla. Ha tenido un impacto enorme en el crecimiento del e-commerce.
- **Spotify API:** Permite acceder a información sobre canciones, artistas, listas de reproducción y usuarios. Es utilizada por aplicaciones de música, análisis de tendencias y dashboards de entretenimiento.

B. Usos y Tecnologías a Utilizar

1. Tecnologías de desarrollo

Las APIs pueden diseñarse bajo distintos paradigmas o estilos arquitectónicos. Los tres más importantes son:

- **REST (Representational State Transfer)**

Es el estilo más común en el desarrollo actual. Utiliza los métodos HTTP (GET, POST, PUT, DELETE) para interactuar con recursos representados en URLs. Los datos suelen intercambiarse en formato JSON.
Ventajas: Fácil de entender, rapida implementacion, altamente compatible con servicios web modernos.
Aplicaciones: Ideal para aplicaciones móviles, sitios web, microservicios y sistemas distribuidos.

- **SOAP (Simple Object Access Protocol)**

Es un protocolo más formal y robusto, basado en XML. Estándar en ambientes corporativos donde la seguridad, integridad y control transaccional son críticos.
Ventajas: Estructura estricta, buena para integraciones empresariales complejas.
Aplicaciones: Banca, salud, seguros, y sistemas que requieren interoperabilidad con múltiples estándares.

- **GraphQL**

Desarrollo por Facebook, es un lenguaje de consulta que permite al cliente especificar exactamente qué datos necesita. Es eficiente y flexible.
Ventajas: Reduce el consumo de datos, mejora la velocidad de las aplicaciones con muchas vistas dinámicas.
Aplicaciones: Aplicaciones móviles, redes sociales, sistemas con interfaces complejas.

Tabla Comparativa

Características	**REST**	**SOAP**	**GraphQL**
Formato de datos	JSON, XML	XML	JSON
Protocolo	HTTP/HTTPS	HTTP, SMTP	HTTP
Flexibilidad	Alta	Baja	Muy alta
Seguridad integrada	Externa (OAuth)	WS-Security	Externa (OAuth)
Facilidad de uso	Alta	Media-baja	Media

2. Lenguajes y herramientas

Las APIs pueden ser desarrolladas y consumidas desde diversos lenguajes y entornos:

- **JavaScript (Node.js):** Muy usado para crear APIs RESTful con frameworks como Express. También permite consumir APIs fácilmente en el navegador.
  Caso: Crear un backend para una app de clima que consuma la API de Open Weather.
- **Python:** Con frameworks como Flask o FastAPI, se crean APIs ligeras y escalables.
  Caso: Automatización de tareas que consumen datos de una API de precios de criptomonedas.
- **Java:** Usado en entornos empresariales. Frameworks como Spring Boot permiten desarrollar APIs robustas.
  Caso: Sistemas bancario que se conecta con otros servicios usando SOAP o REST.
- **PHP, C#, Go:** También son opciones válidas según el tipo de proyecto, empresa o infraestructura.

3. Implementacion practica

Existen varias herramientas que permiten probar, documentar y consumir APIs sin necesidad de escribir código:

- **Postman:** Plataforma muy popular que permite realizar solicitudes a APIs, visualizar respuestas, documentar endpoints, testear seguridad y automatizar pruebas.
- **Insomnia:** Alternativa a Postman, más ligera y centrada en desarrolladores.
- **Swagger (OpenAPI):** Permite documentar APIs REST de forma estandarizada, generar código cliente/servidor y crear interfaces de prueba.
- **cURL (en terminal):** Utilidad de línea de comandos para realizar solicitudes HTTP a APIs.

C. Arquitectura Cliente-Servidor

1. Modelo de comunicación

En el contexto de una API, la arquitectura cliente-servidor es fundamental para comprender cómo interactúan las distintas partes de una aplicación.

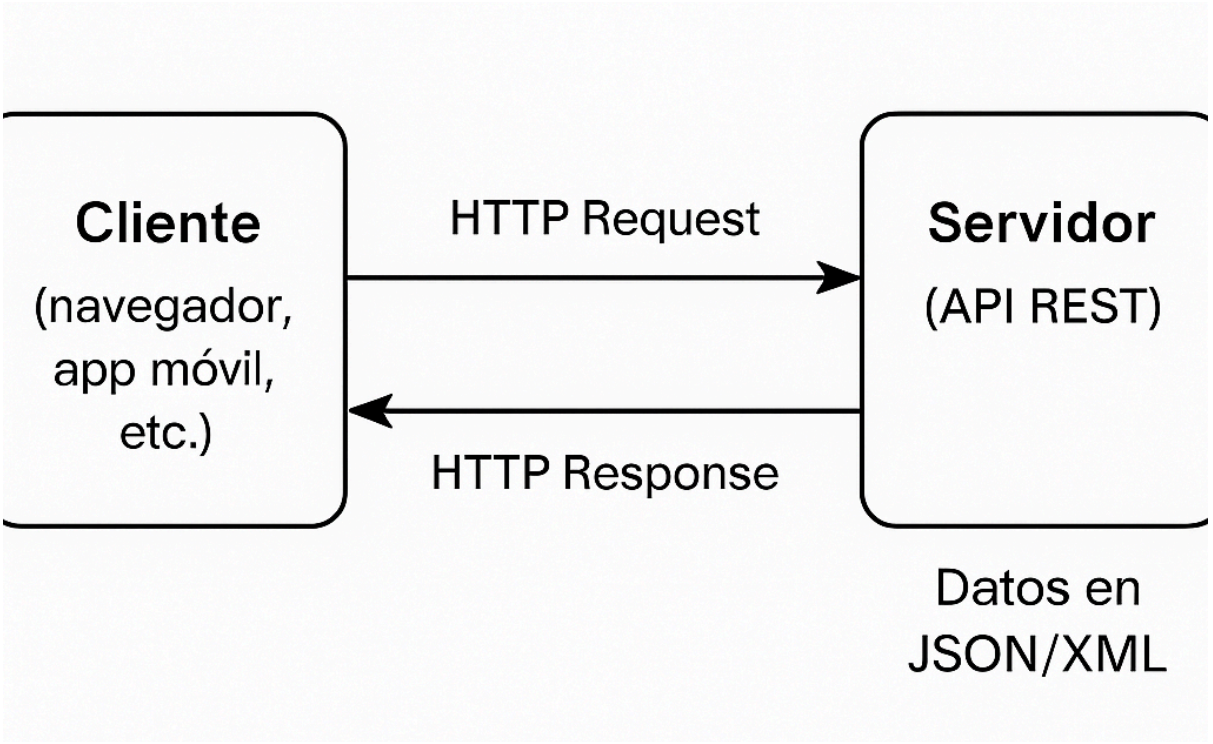
- **Cliente:** Es quien realiza una solicitud para obtener o enviar datos. Pueden ser una app móvil, un navegador web o un sistema de escritorio.
- **Servidor:** Es quien recibe la solicitud, la procesa y devuelve una respuesta, generalmente con datos o un mensaje de confirmación.

Flujo basico de comunicacion:

1. El cliente envía una solicitud HTTP (por ejemplo, GET para pedir datos o POST para enviarlos).
2. El servidor recibe la solicitud, la procesa internamente (puede consultar una base de datos, hacer cálculos, validar datos, etc).
3. El servidor envía una respuesta HTTP con un código de estado (200 OK, 404 Not Found, 500 Error, etc.) y si corresponde, con datos en formatos como JSON o XML.
4. El cliente procesa la respuesta y actualiza su interfaz o toma decisiones según el contenido recibido.

Este modelo permite separar responsabilidades, facilitando el mantenimiento y la escalabilidad del sistema.

2. Diagrama de arquitectura



Opcionalmente, dentro del servidor pueden intervenir:

- Bases de datos (para consultar o guardar información).
- Servicios externos (como otras APIs).
- Sistemas de autenticación (para verificar al usuario).

D. Parámetros de seguridad

La seguridad en el uso de APIs es un aspecto crítico para proteger los datos, garantizar la privacidad de los usuarios y evitar accesos no autorizados o ataques cibernéticos. A continuación, se detallan los principales mecanismos de seguridad y buenas prácticas aplicables en el desarrollo e implementación de APIs.

1. **Requisitos de seguridad**
    a. **Autenticación**

        Es el proceso mediante el cual una API verifica la identidad del cliente que intenta acceder a ella.

- **API Key:** Clave única que identifica a cada consumidor de la API. es simple pero no siempre segura.
- **OAuth 2.0:** Protocolo ampliamente utilizado para permitir que aplicaciones accedan a información sin necesidad de compartir contraseñas. Por ejemplo, iniciar sesión con Google o Facebook.
- **JWT (JSON Web Token):** Permite autenticar usuarios con tokens firmados digitalmente que viajan en cada solicitud.

b. Autorización

Luego de autenticar al usuario, se determina que acciones puede o no realizar dentro de la API. Por ejemplo, un usuario puede tener permiso para leer datos, pero no para modificarlos.

c. Cifrado de datos

- Todas las comunicaciones deben realizarse sobre HTTPS, que cifra los datos intercambiados y protege contra ataques de intermediarios (man-in-the-middle).
- En algunos casos, los datos también se cifran antes de ser almacenados o enviados, agregando una capa adicional de seguridad.

d. Gestión de errores

Las APIs deben manejar correctamente los errores sin exponer información sensible (como rutas internas, detalles de la base de datos, etc.) Ejemplo de buena práctica: retornar errores genéricos como 401 Unauthorized o 500 Internal Server Error.

2. Buenas prácticas

- **Rate Limiting:** Limita la cantidad de solicitudes que un cliente puede hacer en un periodo determinado, para prevenir abusos o ataques de denegación de servicio (DoS).

- **Validación de entradas:** Verificar los datos que envía el cliente para evitar ataques como inyección SQL o ejecución de código malicioso.
- **Registros y monitoreo:** Implementar herramientas de logging para detectar comportamientos sospechosos o intentos de acceso no autorizado.
- **Política de CORS (Cross-Origin Resource Sharing):** Controla qué dominios pueden consumir la API, evitando que sea usada desde fuentes no confiables.
- **Actualización constante:** Mantener las librerías, frameworks y servidores al día, cerrando vulnerabilidades conocidas.

E. Metodologías de desarrollo, Implantación y Uso

La creación y mantenimiento de APIs requiere de un enfoque metodológico bien estructurado que garantice eficiencia, calidad del producto y facilidad de mantenimiento. En esta sección se abordan metodologías de desarrollo, estrategias de despliegue y casos prácticos de implementación.

1. Metodologías de desarrollo

- **Enfoques tradicionales (Modelo en cascada):**
  Se sigue una secuencia lineal de análisis, diseño, desarrollo, pruebas e implementación. Es poco flexible ante cambios, y se usa en proyectos con requisitos muy claros y estables desde el inicio.

- **Metodologías ágiles (Scrum, Kanban):**
  Se trata de un enfoque iterativo e incremental, se trabaja en ciclos cortos (sprints), con entregas frecuentes y validación continua. Fomenta la colaboración entre desarrolladores, diseñadores y usuarios finales. Es muy utilizada en el desarrollo de APIs modernas debido a su adaptabilidad y mejora continua.

2. Proceso de Implantación

Estrategias de despliegue

- **Despliegue continuo (CD)** Automatiza el lanzamiento de nuevas versiones de la API. Se integra con herramientas como GitHub Actions, Jenkins o GitLab CI.
- **Pruebas automatizadas:** Se aplican tests unitarios, de integración y pruebas de carga para asegurar el buen funcionamiento de la API antes del despliegue.
- **Versionado de API:** Permite mantener varias versiones activas para evitar afectar a los clientes que utilizan versiones anteriores.

Mantenimiento continuo

- Monitoreo con herramientas como Prometheus, Datadog o New Relic.
- Análisis de logs para detectar errores o comportamientos anómalos.
- Actualización de documentación (por ejemplo, con Swagger o Postman).

3. Casos prácticos de implantación y uso

a. Ejemplo 1: API de clima

- APIs como OpenWeatherMap permiten integrar información meteorológica en apps móviles.
- Son consumidas desde múltiples plataformas, con control de acceso mediante API Keys.

Diagrama de flujo

[Inicio]
↓
[Usuario ingresa ciudad]
↓
[App envía solicitud GET a la API]
↓
[API válida clave y procesa solicitud]
↓
[Devuelve respuesta con datos del clima]

↓

[App muestra los datos al usuario]

↓

[Fin]

b. **Ejemplo 2: API de pagos (Mercado Pago, Stripe)**
    - Permiten a los negocios incorporar pagos electrónicos de forma segura.
    - Usan OAuth 2.0 para autenticar al usuario y manejan múltiples niveles de autorización.

Diagrama de flujo

[Inicio]

↓

[Usuario elige producto y método de pago]

↓

[App genera token de pago con Stripe]

↓

[Envía token a servidor propio]

↓

[Servidor hace solicitud POST a Stripe API]

↓

[Stripe responde con confirmación o error]

↓

[App muestra resultado al usuario]

↓

[Fin]

c. **Ejemplo 3: API de redes sociales**
    - APIs como la de Instagram o Twitter permiten publicar, leer o analizar contenido.
    - Son ejemplos de APIs altamente versionadas y monitoreadas, con un uso intensivo de control de errores y límites de uso.

Diagrama de flujo

[Inicio]

↓

[Usuario inicia sesión en la app con OAuth (ej. cuenta de Twitter)]

↓

[La app solicita permiso de acceso a la API de Twitter]

↓

[Twitter autoriza y devuelve un token de acceso]

↓

[La app usa el token para hacer una solicitud (GET/POST) a la API]

↓

[La API de Twitter devuelve datos del perfil, publicaciones o confirma publicación]

↓

[La app muestra los datos o notifica acción realizada]

↓

[Fin]

Modelo de madurez de APIs (Richardson Maturity Model)

Este modelo clasifica el nivel de madurez de una API RESTful en 4 niveles:

- Nivel 0: Uso de un único endpoint.
- Nivel 1: Separación por recursos.
- Nivel 2: Uso correcto de métodos HTTP.
- Nivel 3: Implementación de HATEOAS (hipermedia como motor del estado de la aplicación).

Bibliografía

Madden, N. (2020). *API Security in Action*. Manning Publications. https://www.manning.com/books/api-security-in-action

Mozilla. (s.f.). *MDN Web Docs*. https://developer.mozilla.org/es/

Swagger. (s.f.). *Swagger Documentation*. https://swagger.io/docs/

Postman. (s.f.). *Postman Learning Center*. https://learning.postman.com/

free CodeCamp. (s.f.). *Currículo de desarrollo web y APIs*. https://www.freecodecamp.org/

Coursera. (s.f.). *Cursos sobre desarrollo de APIs*. https://www.coursera.org/

edX. (s.f.). *Cursos de APIs y desarrollo web*. https://www.edx.org/

Hoppscotch. (s.f.). *Hoppscotch – Open source API development ecosystem*. https://hoppscotch.io/